

Tema 1 - Agent QA cu Tools + Prompts + ReAct

Autor: Sandu Baboi | Curs: AI Agent Development @ Skillab | Lector: Richard Francu
Data predare: 26 mai 2026 | Sesiunea L2 - Instrumente, prompturi si observabilitate

Repository GitHub (predare prin Git push conform cerintei lectorului)

<https://github.com/sandubaboi-wq/tema-1-qa-agent>

Username Git: sandubaboi-wq | Branch: main | Commit predare: 9910041

Ce am implementat

- Agent ReAct (Reasoning + Acting) cu bucla think -> act -> observe -> repeat (max 10 iteratii).
- Tool Registry cu 3 tool-uri validate prin Pydantic: calculator (ast.parse, fara eval), get_current_datetime (timezone IANA), web_search (mock).
- Prompt Registry cu prompturi YAML + placeholder-uri Jinja2 + singleton lazy via @lru_cache.
- Factory pattern pentru client LLM - suporta 2 provideri (Anthropic, LiteLLM proxy) prin variabila de mediu, fara modificari de cod.
- Catalog tool-uri exportat in 2 formate (OpenAI function calling + Anthropic tool use) pentru compatibilitate cu orice provider.

Pattern-uri din Lectia 2 aplicate

Pattern	Locatie in cod	Slide L2
Prompt Registry	prompts/registry.py	S3
Decorator (@register_tool)	tools/registry.py	S4.3
Factory (create_llm_client)	agent.py	S4.4
Singleton lazy (@lru_cache)	prompts/registry.py	S4.6
Tool Registry + Wrapper	tools/	S7
ReAct loop	agent.py:react_loop	S6

Structura proiect

```
tema_1/  
  prompts/  
    react_system.yaml      # Prompturile sunt CONFIGURATIE, nu cod  
    registry.py            # System prompt cu placeholder-uri Jinja2  
                           # PromptRegistry (singleton lazy)  
  tools/  
    params_models.py       # Tool Registry  
                           # Pydantic BaseModel per tool  
    registry.py            # TOOL_REGISTRY + @register_tool  
    basic_tools.py         # calculator, datetime, web_search  
    tool_wrapper.py        # ToolWrapper.call() + catalog  
  agent.py                 # Bucla ReAct + factory client LLM  
  test_prompts.py          # 3 verificari PromptRegistry  
  test_tools.py            # 6 verificari Tools cu Pydantic  
  test_agent.py            # 5 scenarii end-to-end  
  README.md / requirements.txt
```

Rulare

```
python -m venv venv  
venv\Scripts\activate  
pip install -r requirements.txt  
  
# Provider Anthropic  
set LLM_PROVIDER=anthropic  
set ANTHROPIC_API_KEY=sk-ant-...  
  
# SAU provider LiteLLM (gratuit, local)  
set LLM_PROVIDER=litellm  
  
python agent.py                # mod interactiv CLI
```

```
python test_prompts.py      # test PromptRegistry
python test_tools.py        # test Tools cu Pydantic
python test_agent.py        # test Agent end-to-end
```

Anti-patterns evitate (robustete)

- eval() pentru calculator inlocuit cu ast.parse + whitelist operatori (siguranta).
- Loop infinit la tool calls: detectie repetitie identica + max_iterations=10.
- Erori raw spre LLM: toate erorile sunt capturate si transformate in dict {success, error} cu mesaj human-readable.
- Tool-urile NU arunca exceptii - returneaza mereu dict, agentul decide pasul urmator.

Predat conform cerintei lectorului prin Git push pe repo privat. Acest PDF este dovada operationala pentru LMS.